

AI 데이터 분석 · 2일차

파이썬 기초 + Git 첫걸음

출력 · 연산 · 오류 · 변수, 그리고 버전 관리의 시작

코드 vs 결과 · 오류는 읽는 정보 · Git으로 기록 남기기

VS Code · 파이썬 파일(.py) · 터미널 · Git · GitHub

어제는 코랩에서 데이터 분석, 오늘은 내 컴퓨터에서 코드와 버전 관리

2026-07-14 AI 데이터 분석 교육 정리 (2일차)

학습자 Minho · 국립금오공과대학교 컴퓨터공학부

이 책을 읽는 법

어제 책과 같은 방식입니다. 오늘 직접 만든 다섯 개의 파이썬 파일(01_print.py ~ 04_variable.py, example.py)과 세 개의 강의 교안(변수 / print·오류 / Git)을 하나로 엮었습니다. 오늘은 분량이 많아, 코드 하나하나를 "무엇인지 (분류)"부터 짚고 갑니다.

분류 뱃지

내장함수 print·abs·str **연산자** + - * // % **자료형** 숫자·문자 **키워드** 예약어

문법 f-string·재할당 **명령어** git · 터미널 **개념** 코드 vs 결과 등

코드블록 두 가지

파란 세로줄 = 파이썬 코드(.py), **회색 세로줄 + TERMINAL** = 터미널에 치는 명령어(git 등). 초록 ▶ 줄은 실행 결과입니다.

개념

오늘의 큰 그림: ① 내 컴퓨터(VS Code)에서 파이썬 파일을 쓰고 → ② **print로 확인하고** → ③ **오류를 읽어 고치고** → ④ **변수로 값을 다루고** → ⑤ **Git으로 그 기록을 남긴다**. 이 다섯 줄기가 오늘 배운 전부입니다.

목차

	장 / STEP	무엇을 배우나
0장	오늘의 용어 다지기	코드 vs 결과 · 함수와 괄호 · 숫자와 문자
STEP 1	print — 화면에 찍어 확인하기	내장함수 · 따옴표 · str() · f-string · 여러 값 출력
STEP 2	연산자로 계산하기	+ - * // % · 연산 순서 · 나누기와 실수 · abs()
STEP 3	주석과 편집 단축키	# 주석 · Ctrl+/ · Alt+ ↑ ↓ · 저장/실행
STEP 4	오류는 읽는 정보다	Traceback · NameError · SyntaxError · 대처 4단계
STEP 5	변수 — 값에 이름 붙이기	할당 · 재할당 · 값 복사 · 이름 규칙 · snake_case
STEP 6	Git & GitHub — 기록을 남기다	버전관리 · add/commit · 로컬vs원격 · push/pull/clone
부록 A	오늘의 명령어·함수 치트시트	파이썬 + Git 한눈에
부록 B	오류 사전 & 자주 하는 실수	NameError·SyntaxError + 오늘 코드의 함정
부록 C	강의 흐름 연결 & 다음 시간 예고	자료형·type()·input은 다음 시간

CHAPTER 0

오늘의 용어 다지기

코드를 쓰기 전에, 오늘 나올 말들부터 정확히

어제는 데이터 분석 도구(pandas·matplotlib)를 배웠고, 오늘은 그 아래에 깔린 "코드의 기본기"와 개발자의 필수 도구 "Git"을 배웁니다. 먼저 오늘 계속 등장할 용어부터 잡고 갑니다.

0-1. 코드 vs 결과 — 가장 먼저 구분할 것

강사님이 반복해서 강조한 핵심입니다. 내가 타이핑한 것은 코드, 실행 후 저절로 나타난 것은 결과입니다.

	무엇인가	바꾸는 방법
코드	내가 직접 타이핑하는 명령	직접 고쳐 쓴다
결과	실행한 뒤 화면(터미널)에 나타남	결과가 아니라 "코드"를 고친다

개념

컴퓨터는 코드를 위에서 아래로, 한 줄씩, 글자 그대로 읽습니다. "의도"가 아니라 "글자"를 받아들이기 때문에 글자 하나·순서 하나가 결과를 바꿉니다. 그래서 처음엔 오류가 잦은 게 자연스럽습니다.

0-2. 함수와 괄호 — 자판기로 이해하기

print 같은 것을 함수라고 부릅니다. 강사님의 비유가 명쾌합니다.

비유

자판기 를 떠올려 보세요.

함수 = 자판기 그 자체 (안이 어떻게 생겼는지 몰라도, 넣으면 결과를 내주는 도구)

괄호 () = 동전 넣는 입구 (재료·값을 넣는 자리)

괄호 안의 값 = 넣는 동전 (이 값을 인자(argument)라고 부름)

그래서 print("안녕")은 "print라는 도구에 '안녕'이라는 재료를 넣어 화면에 찍어라"라는 뜻입니다. 괄호를 빼먹으면 컴퓨터가 이게 함수인지 몰라 오류가 납니다.

0-3. 오늘 나오는 분류 한눈에

분류	무엇인가	오늘의 예시
내장함수	파이썬에 기본 내장된 도구(직접 안 만들어도 씬)	<code>print()</code> <code>abs()</code> <code>str()</code>
자료형	값의 종류 — 숫자와 문자	<code>75</code> <code>2.3</code> <code>"센서 A"</code>
연산자	계산·비교 기호	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>//</code> <code>%</code> <code>==</code>
키워드	파이썬 예약어(이름으로 못 씬)	<code>if</code> <code>for</code> <code>True</code>
문법	f-string·재할당 같은 표현 방식	<code>f"{temp}"</code> <code>x = x + 5</code>
명령어	터미널·Git에 치는 명령	<code>git add</code> <code>.</code> <code>git commit</code>

주의

예약어 구분 팁(강사님): VS Code에서 **자동으로 색이 입혀지는 단어**(`if`·`for`·`True`·`print` 등)는 변수 이름으로 쓰지 않습니다. 색이 안 입혀진 단어만 내 변수 이름으로 씬니다.

STEP 1

print — 화면에 찍어 확인하기

01_print.py · 내장함수와 출력의 모든 것

내장함수 print() 터미널에 값을 출력

정의 괄호 안의 값을 화면(터미널)에 찍어내는, 파이썬 기본 내장 함수. print = 영어로 "찍어내다".

형태 print(값)

```
print("안녕하세요")    # 안녕하세요
print(714)             # 714
print("첫 번째 줄")
print("두 번째 줄")
```

```
▶ 안녕하세요
▶ 714
▶ 첫 번째 줄
▶ 두 번째 줄
```

왜 쓰나 파이썬은 시키지 않으면 아무것도 보여주지 않습니다. 데이터가 잘 들어왔는지, 계산이 맞는지 확인할 때마다 print로 찍어 봅니다. print 한 번이 한 줄입니다.

개념

어제와 같은 원칙: 실행했는데 아무것도 안 나와도 대개 정상입니다. 결과를 보려면 print로 직접 출력하세요. print 는 "확인·디버깅의 도구"입니다.

자료형 숫자 vs 문자 (따옴표의 차이) 오늘의 가장 중요한 구분

정의 따옴표로 감싸지 않으면 숫자(계산 가능한 진짜 값), 따옴표로 감싸면 문자(글자 모양)로 취급합니다.

```
print(1134)           # 숫자 1134
print(12.34)          # 소수점도 그대로
print(1)              # 숫자 1
print('1')            # 문자 '1' (똑같아 보여도 다름!)
```

```
▶ 1134
▶ 12.34
▶ 1
▶ 1
```

왜 쓰나 설비 측정값(75, 2.3)은 계산해야 하니 따옴표 없이, 항목 이름·설명은 글자니까 따옴표로. 같은 75라도 따옴표가 있으면 글자, 없으면 숫자입니다.

주의

`print(1)`과 `print('1')`은 화면상 똑같이 1로 보이지만 종류가 다릅니다. 숫자는 계산되고, 문자는 글자 그대로 다뤄집니다.

문법 문자열 따옴표 규칙 큰따옴표 · 작은따옴표

정의 글자는 반드시 따옴표로 감쌉니다. 큰따옴표 `" "` 든 작은따옴표 `' '` 든 "한 쌍이 같으면" 됩니다. 혼용은 금지.

```
print("온도 측정값")    # OK
print('진동 측정값')    # OK
print("잘못 섞기")      # x 앞뒤가 다르면 오류
```

왜 쓰나 따옴표는 컴퓨터에게 "이 한 쌍 안은 글자야"라고 알려주는 표시입니다. 따옴표를 빼면 그 글자를 "어떤 값을 가리키는 이름(변수)"으로 착각합니다.

자주 하는 실수

대소문자도 구분합니다. `Print()`(대문자 P)는 동작하지 않아요. 코드는 대문자·소문자를 다른 글자로 봅니다.

연산자 문자열의 `+` 와 `*` 글자도 연산은 되지만 계산은 아님

정의 숫자의 `+`, `*`는 계산이지만, 문자의 `+`는 이어붙이기, `*`는 반복입니다.

```
print(123 + 456)        # 579    (숫자 계산)
print("123" + "456")    # 123456 (글자 이어붙이기)
print(5 * 2)             # 10     (숫자 계산)
print("5" * 2)           # 55     (글자 반복)
```

- ▶ 579
- ▶ 123456
- ▶ 10
- ▶ 55

왜 쓰나 "글자는 연산이 아예 안 된다"는 틀린 말입니다. `+`로 잇고 `*`로 반복은 되지만, 숫자처럼 계산해 주진 않습니다. (문자끼리 빼기·나누기는 안 됨)

내장함수 `str()` 숫자를 문자로 형변환

정의 값을 문자(문자열)로 바꿔주는 내장 함수(형변환 도구).

형태 `str(숫자)`

```
temperature = 75
# print("온도: " + temperature)    # x 문자 + 숫자 → 오류
print("온도(°C): " + str(temperature))  # ✓ 숫자를 문자로
```

▶ 온도(°C): 75

왜 쓰나 +로 글자와 숫자를 이으려면 종류를 맞춰야 합니다. 숫자를 `str()`로 문자로 바꾸면 이어붙일 수 있습니다. (다음 시간의 "형변환"을 미리 맛보기)

문법 `f-string` `f"...{ }..."` 문장 안에 값 끼우기

정의 문자열 앞에 `f`를 붙이고 `{ }` 안에 값·변수를 넣으면 그 자리에 값이 채워집니다. (어제 배운 것 복습)

```
print(f"오늘 날짜 숫자로: {20260714}")
hello = "안녕하세요"
print(f"인사: {hello}")
```

▶ 오늘 날짜 숫자로: 20260714
▶ 인사: 안녕하세요

왜 쓰나 `str()`로 일일이 바꾸고 +로 잇는 것보다 훨씬 깔끔하게 문장에 값을 넣습니다.

주의

`{ }` 안에 따옴표를 겹쳐 쓸 때 주의(파이썬 3.12 미만 오류). 바깥이 큰따옴표면 안쪽은 작은따옴표로:
`f"{a['key']}"`.

문법 여러 값 출력: `쉼표(,)` vs `더하기(+)` 한 줄에 여러 값

정의 괄호 안에서 값을 쉼표로 나열하면 사이에 공백 1칸이 자동으로 들어갑니다. +로 이으면 공백 없이 붙습니다 (문자만).

```
temperature = 75
print("온도", temperature)          # 쉼표 → 공백 자동
print("온도(°C): " + str(temperature))  # + → 공백 없음, str 필요
```

▶ 온도 75

▶ 온도(°C): 75

왜 쓰나 숫자 75만 찍으면 무슨 값인지 알 수 없습니다. "온도"라는 이름을 함께 출력하면 나중에 버그를 찾을 때 훨씬 편합니다.

개념

침표(.)는 숫자·문자를 섞어도 되고 공백을 자동으로 넣어 편합니다. 더하기(+)는 공백이 없고 문자끼리만 되며 숫자는 `str()`가 필요합니다. 상황에 맞게 고릅니다.

STEP 2

연산자로 계산하기

02_operator.py · 사칙연산과 그 너머

연산자 + - * / // % 산술 연산자 6종

정의 더하기·빼기·곱하기·나누기, 그리고 몫(//)과 나머지(%).

```
print(7 + 14)    # 21
print(7 - 14)    # -7
print(7 * 14)    # 98
print(7 / 14)    # 0.5
print(7 // 14)   # 0    (몫)
print(7 % 14)    # 7    (나머지)
```

```
▶ 21
▶ -7
▶ 98
▶ 0.5
▶ 0
▶ 7
```

왜 쓰나 어제 배운 대로 데이터는 모두 숫자입니다. 평소와의 차이(빼기)·평균(나누기) 계산이 분석의 기본입니다.

연산자 연산 순서와 괄호 () 계산 순서 지정

정의 곱하기·나누기가 더하기·빼기보다 먼저입니다. 괄호로 순서를 바꿀 수 있습니다.

```
print(75 + 80 / 4)    # 95.0  (80/4 먼저 = 20, +75)
print((75 + 80) / 4)  # 38.75 (괄호 먼저 = 155, /4)
```

```
▶ 95.0
▶ 38.75
```

왜 쓰나 평균을 구하려면 "먼저 더하고 나서 나눠야" 합니다. 괄호로 (합)/개수 순서를 강제합니다. 수학의 괄호와 같습니다.

개념 나누기(/)는 결과가 실수(float) 소수점이 붙는 이유

정의 나누기 연산자를 쓰는 순간 결과가 실수로 바뀝니다. 딱 떨어져도 뒤에 .0이 붙습니다.

```
print(8 / 4)    # 2.0  (딱 떨어져도 .0)
print(7 + 14)   # 21   (정수 그대로)
```

- ▶ 2.0
- ▶ 21

왜 쓰나 정수(21)와 실수(2.0)를 눈으로 구분할 수 있어야, 나중에 계산 결과의 소수점을 보고 당황하지 않습니다. 뭇만 필요하면 //를 씁니다.

내장함수 `abs()` 절댓값(부호 없는 크기)

정의 괄호 안 숫자의 절댓값을 돌려주는 내장 함수. 음수도 양수 크기로.

형태 `abs(숫자)`

```
Temperature_1st = 75
Temperature_2nd = 80
print("온도 변화량:", abs(Temperature_1st - Temperature_2nd))
```

▶ 온도 변화량: 5

왜 쓰나 온도 "변화량"은 방향이 아니라 크기입니다. $75-80=-5$ 든 $80-75=5$ 든 변화량은 5. `abs()`로 항상 양수 크기를 얻습니다.

STEP 3

주석과 편집 단축키

메모 남기기 · VS Code를 손에 익히기

문법 주석 # 사람을 위한 메모

정의 샵(#) 뒤에 적은 내용은 컴퓨터가 실행하지 않습니다. 코드가 아니라 사람이 읽는 설명입니다.

```
temp = 25    # 온도(섭씨)
count = 3    # 센서 개수
# temp = 100  # 줄 앞에 #을 붙이면 그 줄은 잠깐 꺼짐(실행 안 함)
```

왜 쓰나 "미래의 나"를 위한 메모입니다. 3일만 지나도 내 코드가 왜 이렇게 흘렀는지 기억이 안 납니다. 이해한 내용을 주석으로 남겨야 나중에 다시 읽힙니다.

주의

오류가 나는 코드를 지우지 말고 주석 처리해 두세요. 지우면 기억이 안 나고, 남겨두면 복습이 됩니다(강사님 방식).

편집·실행 단축키 (VS Code)

어제의 코랩 단축키와 다릅니다. 오늘부터는 내 컴퓨터의 VS Code라서 아래 단축키를 손에 익히면 훨씬 빨라집니다.

단축키	기능
Ctrl + /	선택한 줄 주석 토글(닫기 ↔ 풀기)
Alt + ↑ / ↓	커서가 있는 줄을 위/아래로 이동
Tab	자동완성 제안 수락(화살표로 고르고 Tab/Enter)
Ctrl + S	저장 — 탭의 ● 표시가 사라지면 저장됨
Ctrl + Z / Ctrl + Shift + Z	되돌리기 / 다시 실행(앞으로 가기)
F5 / Ctrl + F5	파이썬 파일 실행

주의

실행했는데 결과가 이상하거나 안 바뀌면, **저장 안 됨**일 가능성이 높습니다. 파일 탭에 ● (까만 동그라미)가

있는지 확인하고 Ctrl+S를 누르세요. 그러면 옛날 결과가 아니라 지금 코드가 실행됩니다.

STEP 4

오류는 읽는 정보다

03_error.py · 빨간 메시지를 두려워하지 않기

개념

오류는 실패가 아니라 **과정**입니다. 현직 개발자도 하루에 수십 번 오류를 만나고, 코드 쓰는 시간보다 오류 읽는 시간이 더 길 때도 있습니다. 오류를 내는 건 이상한 게 아니라, **핵심만 빨리 읽고 고치는 것이** 중요합니다.

개념 Traceback 읽는 법 맨 아래가 진짜 에러

정의 오류 메시지(Traceback)는 특이하게 "가장 아래쪽"이 진짜 에러가 난 위치입니다. 위→아래로 갈수록 실행된 시간 순서입니다.

TERMINAL — 오류 메시지

```
Traceback (most recent call last):
  File "main.py", line 8, in <module>
    order_coffee()
  File "main.py", line 2, in make_coffee
    print(10 / 0)
ZeroDivisionError: division by zero ← 진짜 원인은 맨 아래!
```

왜 쓰나 길고 복잡해 보여도, 딱 세 가지만 봅니다 → ① 위치(line 몇 번째 줄), ② 종류(ZeroDivisionError 등), ③ 설명 (division by zero). 나머지는 안 읽어도 됩니다.

개념 NameError 정의되지 않은 이름

정의 만든 적 없는 이름을 불렀을 때 나는 오류. "그 이름을 못 찾겠다"는 뜻.

TERMINAL

```
print(온도)
# NameError: name '온도' is not defined
```

왜 쓰나 따옴표를 빠뜨리면(온도) 컴퓨터는 그걸 변수 이름으로 찾습니다. 그런 변수가 없으니 NameError. is not defined = "정의되지 않음".

자주 하는 실수

가장 흔한 원인 4가지: ① 따옴표 누락(글자인데 따옴표 안 씌), ② 변수 이름 오타, ③ 대소문자 불일치(temp ≠

Temp), ④ 변수 만들기 전에 사용.

개념 `SyntaxError` 문법이 어긋남

정의 문법(약속)을 어겼을 때 나는 오류. 따옴표나 괄호 짝이 안 맞을 때 자주 납니다.

```
# print("Hello World'    ← 앞뒤 따옴표가 다름
# print("진동)           ← 괄호를 안 닫음
# print("압력": 4.0)      ← 콜론을 잘못 씀
```

왜 쓰나 문법은 한글 맞춤법 같은 "정해진 약속"입니다. 컴퓨터는 문법을 먼저 검사하고, 통과해야 그 다음에 변수를 찾습니다. 그래서 문법 오류가 있으면 변수 문제는 보이지도 않습니다.

주의

invalid syntax / never closed / unterminated string 같은 설명이 보이면 "따옴표·괄호 짝"부터 확인하세요.

오류 대처 4단계

단계	할 일
① 당황하지 않기	오류는 정상 과정. 빨간 글씨 = 안내문
② 읽기	맨 아랫줄의 종류 + 위치(line) 찾기
③ 이해하기	무슨 뜻인지 파악(모르면 번역기)
④ 검색하기	오류 종류+설명을 그대로 검색(영어로!)

주의

실전 팁: 오류는 **영어로 구글링**하는 게 빠릅니다(자료가 훨씬 많음). 터미널 복사가 안 되면 캡처해서 번역기·GPT에 넣되, "**해석 먼저**" 하고 **스스로 이해**한 뒤 고치세요. 매번 AI를 켜 줄 순 없고, 자주 나오는 오류는 직접 고치는 게 더 빠릅니다.

개념 실습 — 오류 세 줄 고치기 (`03_error.py`) 직접 디버깅

정의 일부러 낸 오류를 하나씩 고쳐 봅니다. 위에서 오류가 나면 그 아래는 실행되지 않으므로, 하나씩 차근차근.

```
# 고치기 전 (주석 처리해 둔 오류)
```

```
# print(온도, 75)      → 따옴표 없음
# print("진동", 2.3)   → 괄호 안 닫음
# print("압력": 4.0)   → 콜론이 잘못됨
```

고친 뒤

```
print("온도", 75)      # 따옴표 추가
print("진동", 2.3)     # 괄호 닫기
print("압력", 4.0)     # 콜론을 쉼표로
```

▶ 온도 75
▶ 진동 2.3
▶ 압력 4.0

왜 쓰나 디버깅(debugging)은 버그(bug)를 잡는 일. 여러 오류가 섞여도 한 번에 하나씩만 나타나니, 순서대로 고치면 언젠가 끝납니다.

STEP 5

변수 — 값에 이름 붙이기

04_variable.py · example.py · 오늘의 핵심

개념 변수란? (이름표 붙은 상자) 값에 붙이는 이름

정의 변수는 값에 붙인 이름입니다. 이름표가 붙은 상자에 값을 담아두고, 이름으로 다시 꺼내 씁니다.

```
temp = 36      # 숫자는 따옴표 없이
name = "센서 A" # 글자는 따옴표로
print(temp)    # 36
print(name)    # 센서 A
```

▶ 36

▶ 센서 A

왜 쓰나 75도를 매번 외우는 대신 temp라는 이름에 담아두면, 이름만 보고 "이건 온도구나" 알 수 있고 값이 바뀌어도 한 곳만 고치면 됩니다. 숫자·문자·참거짓을 담습니다.

개념

비유: 이름표는 그대로 두고, 상자 안의 값만 바꿉니다. temp라는 이름표 상자에 25가 들었다가 30으로 바뀌는 식입니다.

연산자 = 는 "저장", == 는 "비교" 할당과 비교의 차이

정의 =는 "같다"가 아니라 "오른쪽 값을 왼쪽 이름에 저장하라"는 명령(할당). 같은지 비교는 등호 두 개 ==.

```
temp = 25      # temp에 25를 저장(할당)
# temp == 25   # temp가 25와 같은지 비교(참/거짓)
```

왜 쓰나 변수를 "선언"(이름 만들기)하고 값을 "할당"(저장)하는 게 오늘의 기본 동작입니다. =를 "같다"로 읽으면 $x = x + 5$ 같은 코드가 이해가 안 됩니다.

개념 실행 순서와 "가장 가까운 값" 위에서 아래로

정의 코드는 위→아래로 실행됩니다. print(temp)를 만나면 위로 올라가 "가장 가까운" temp 할당을 찾아 그 값을 씁니다.

```
x = 10
print(x)    # 10
x = 20
print(x)    # 20 (가장 가까운 위쪽 값)
```

```
▶ 10
▶ 20
```

왜 쓰나 값이 이상할 때, 대개 "재할당이 반영됐거나" "재할당을 빠뜨린" 경우입니다. 실행 순서를 따라가면 원인이 보입니다.

자주 하는 실수

print(x)를 맨 위(x를 만들기 전)에 두면 NameError. "아직 x를 안 만들었다"는 뜻입니다.

문법 재할당 $x = x + 5$ · += 자기 값을 갱신

정의 오른쪽을 먼저 계산한 뒤 그 결과를 다시 왼쪽에 저장합니다. $x += 20$ 은 $x = x + 20$ 의 줄임.

```
x = 10
x = x + 5    # 오른쪽 10+5=15 → 다시 x에
print(x)    # 15
x = x * 2
print(x)    # 30
temp = 15
temp += 20  # temp = temp + 20
print(temp) # 35
```

```
▶ 15
▶ 30
▶ 35
```

왜 쓰나 $x = x + 5$ 가 오류가 아닌 이유: =는 "같다"가 아니라 "저장"이라서, 오른쪽을 먼저 계산해 다시 넣는 것입니다. 마지막 할당만 최종 값으로 남습니다(10→20→30이면 30).

개념 값 복사 $b = a$ 그 순간의 값만 복사

정의 $b = a$ 는 그 순간 a의 값을 복사할 뿐, 두 변수를 묶는 게 아닙니다. 이후 a가 바뀌어도 b는 그대로.

```
a = 100
b = a        # 지금 a의 값(100)을 b에 복사
a = 999      # a만 바꿈
print(a)     # 999
print(b)     # 100 (b는 복사 시점 값 유지!)
```

```
▶ 999
▶ 100
```

왜 쓰나 많은 사람이 b도 999를 예상하지만 정답은 100. 원본을 보존하거나 값을 안전하게 옮길 때 이 성질을 이용합니다.

문법 여러 변수 한 번에 `x, y, z = 1, 2, 3` 다중 할당

정의 쉼표로 이름과 값을 나열하면 순서대로 짝지어 저장됩니다.

```
x, y, z = 1, 2, 3    # x=1, y=2, z=3
width, height = 4, 3
print(width)        # 4
print(height)       # 3
```

```
▶ 4
▶ 3
```

왜 쓰나 변수를 여러 개 만들거나 값을 서로 바꿀 때 유용합니다. 단, 이름 개수와 값 개수가 같아야 합니다.

자주 하는 실수

`x, y, z = 36, 38` 처럼 개수가 안 맞으면 오류. 파이썬은 빈 값을 자동으로 넣어주지 않고 "짝이 안 맞아"라며 멈춥니다.

개념 변수 이름 규칙 되는 이름 / 안 되는 이름

정의 영문자·숫자·밑줄(_)만 사용. 숫자로 시작 불가, 공백 불가, 대소문자 구분.

```
temp2 = 1          # OK (숫자로 시작 아님)
# 2temp = 1        # x 숫자로 시작
temp = 25
Temp = 100          # temp와 Temp는 서로 다른 변수!
print(temp)         # 25
print(Temp)         # 100
```

```
▶ 25
▶ 100
```

왜 쓰나 이름 규칙을 어기면 `SyntaxError`, 대소문자를 헛갈리면 `NameError`가 납니다. 두 단어는 공백 대신 밑줄로 있습니다.

키워드 예약어 & 피해야 할 이름 `if` · `for` · `True` · `print`

정의 `if`·`for`·`True` 같은 예약어는 변수 이름으로 쓸 수 없습니다. `print`·`type` 같은 함수 이름을 덮어쓰는 것도 금지.

```
# if = 3          # x 예약어(조건문)라 이름 불가
```

```
# for = 5      # x 예약어(반복문)
# print = 1    # x print를 덮어쓰면 나중에 출력이 이상해짐
```

왜 쓰나 if는 조건문, for는 반복문에 이미 쓰는 단어입니다. 컴퓨터가 "이게 변수인지 문법인지" 헷갈리지 않게 막아둔 것입니다.

주의

VS Code에서 색이 입혀지는 단어(보통 보라색 등)는 예약어라고 생각하고 이름으로 쓰지 마세요. 변수는 대체로 색이 안 입혀집니다.

개념 좋은 이름 — snake_case 동작하는 이름 ≠ 좋은 이름

정의 a·b·x1도 동작은 하지만, 이름만 봐도 뜻이 드러나는 이름이 좋은 이름입니다. 파이썬은 소문자+밑줄(snake_case)이 표준.

```
# 아쉬운 예
a = 25
b = 3
# 좋은 예
device_temp = 25    # 뱀처럼 밑줄로 이음
sensor_count = 3
print(device_temp)  # 25
▶ 25
```

왜 쓰나 이름만 봐도 "온도·센서 개수"임이 드러나 협업·유지보수가 쉽습니다. 여러 단어는 밑줄로 잇고 소문자로 통일합니다(snake_case).

STEP 6

Git & GitHub — 기록을 남기다

오늘의 마지막, 그리고 취업에 가장 중요한 도구

개념 버전 관리란? (파일명 지옥 탈출) 변경 이력을 기록·관리

정의 파일·코드의 변경 이력을 기록하고 관리하는 시스템. 언제·누가·무엇을·왜 바꿨는지 추적하고, 이전 버전으로 되돌릴 수 있습니다.

```
project.html
project_backup.html
project_final.html
project_진짜최종.html
project_진짜최종_v2.html    ← 어느 게 최신?!
```

왜 쓰나 파일명에 backup·final·진짜최종을 붙이다 보면 폭발합니다. ① 어느 게 최신인지, ② 무엇이 왜 바뀌었는지, ③ 여러 명이 고치면 누구 게 맞는지 알 수 없습니다. 이 세 한계를 버전 관리가 해결합니다.

개념 Git = 스냅샷 (사진 앨범 비유) 그 시점 전체를 기록

정의 Git은 저장할 때마다 그 시점의 전체 상태를 "스냅샷(사진)"으로 남기는 버전 관리 시스템입니다.

왜 쓰나 스마트폰 사진 앨범처럼, 커밋할 때마다 ① 언제 ② 누가 ③ 무엇을 바꿨는지가 남습니다. 언제든 그 시점으로 되돌릴 수 있습니다.

개념

폴더 복사 백업과의 차이: 변경 내용을 줄 단위로 자동 추적하고, 변경 이유를 커밋 메시지로 남기며, 전체가 아니라 "변경분만" 저장하고, 명령어 하나로 과거로 되돌립니다.

개념 왜 배우나 — 포트폴리오 개발자의 실력 증명

정의 Git과 연동한 GitHub의 활동 기록과 개인 저장소는, 개발자의 실력을 보여주는 가장 좋은 자료입니다.

왜 쓰나 꾸준한 커밋 기록·프로젝트 저장소가 곧 이력서가 됩니다. 백엔드·임베디드·DX/AX 어느 방향이든, GitHub는 취업에서 강력한 무기입니다.

개발 환경 설정 (3가지 준비)

도구	무엇 / 어디서
GitHub 가입	github.com — 원격 저장소를 올려두는 플랫폼
Git 설치	Windows: git-scm.com / Mac: brew install git
VS Code 확장	GitLens(활동 시각화), Auto Commit Message

명령어 git config (첫 1회 설정) 나를 등록

정의 Git 첫 설치 후 딱 한 번, 내 이름과 이메일을 등록합니다. 커밋에 "누가"를 기록하기 위함.

```
TERMINAL
git config --global init.defaultBranch main
git config --global user.name "프로필 이름"
git config --global user.email "이메일 주소"
git config --global --list # 설정 확인
```

왜 쓰나 앞으로 남길 모든 커밋에 "누가 언제" 했는지 붙습니다. --global은 이 컴퓨터 전체에 한 번만 하면 된다는 뜻.

명령어 git init & .git 폴더 폴더를 저장소로

정의 프로젝트 폴더에서 딱 한 번 실행해, 그 폴더를 Git 저장소로 만듭니다. 숨김 폴더 .git/이 생기고 여기에 모든 이력이 저장됩니다.

```
TERMINAL
git init
# 실행하면 폴더 안에 숨김 폴더 .git/ 생성
ls -a # .git 확인(숨김 폴더 보기)
```

왜 쓰나 Git은 폴더별로 이력을 관리합니다. .git 폴더가 곧 저장소 그 자체입니다.

자주 하는 실수

.git 폴더는 절대 수동으로 삭제·수정하지 마세요. 삭제하면 지금까지의 모든 히스토리가 복구 불가능하게 사라집니다.

작업 흐름 — 파일의 네 가지 상태

Git에서 파일은 네 상태를 오갑니다. 이 흐름이 Git의 심장입니다.

상태	의미	다음 명령
Untracked	Git이 아직 모르는 새 파일	<code>git add</code>
Modified	추적 중인 파일이 수정됨	<code>git add</code>
Staged	add 후 — 다음 커밋에 포함될 준비	<code>git commit</code>
Committed	commit 후 — 스냅샷이 이력에 저장됨	완료

개념

왜 add → commit 두 단계일까? 10개 파일을 고쳤어도 이번 커밋엔 3개만 넣고 싶을 때가 있습니다. 원하는 것만 add해서 Staged로 올리고, commit으로 그 3개만 스냅샷을 남깁니다. "고르는 단계(add)"와 "확정 단계(commit)"를 나누는 것입니다.

명령어 `git add` · `git commit` 올리고, 스냅샷 찍기

정의 add는 변경 파일을 Staged로 올리고, commit은 그것을 스냅샷으로 확정해 이력에 남깁니다.

TERMINAL

```
git add hello.txt      # 특정 파일만 올리기
git add .              # 현재 폴더의 모든 변경 올리기
git commit -m "notes.txt 추가"  # 스냅샷 + 메시지
```

왜 쓰나 -m 옵션으로 커밋 메시지를 한 줄로 바로 답니다. 이 메시지가 "무엇을 왜 바꿨는지"의 기록이 됩니다.

개념 좋은 커밋 메시지 한 달 뒤의 나도 알 수 있게

정의 "동사 + 대상" 형식으로, 무엇을 했는지 명확하게.

TERMINAL

```
# x 나쁜 예
git commit -m "수정"
git commit -m "ㅇ"

# ✓ 좋은 예
git commit -m "notes.txt 추가"
git commit -m "로그인 버튼 오류 수정"
```

왜 쓰나 "수정/fix/○"은 나중에 무엇을 했는지 알 수 없습니다. 한 달 뒤에도 맥락을 알 수 있게 구체적으로 씁니다.

명령어 `git status` · `git log` 상태와 이력 확인

정의 status는 현재 상태(색으로 구분), log는 지금까지의 커밋 이력을 보여줍니다.

TERMINAL

```
git status    # 초록=Staged, 빨강=수정됐지만 add 안 함
git log       # 커밋 이력(시간 역순), q 키로 종료
```

왜 쓰나 status로 "지금 무엇이 올라가 있나"를, log로 "지금까지 무엇을 남겼나"를 확인합니다. 커밋 전 습관적으로 status를 봅니다.

개념 `.gitignore` - 추적 제외 올리면 안 되는 파일

정의 프로젝트 루트에 두는 텍스트 파일. 여기 적은 파일을 Git이 무시합니다.

```
.env          # API 키·비밀번호 (절대 공개 금지!)
.DS_Store     # 맥 시스템 파일
node_modules/ # 라이브러리 폴더(수천 개)
*.log         # 로그 파일
```

왜 쓰나 .env에 담긴 비밀번호가 GitHub에 올라가면 전 세계에 공개됩니다. 시스템 자동 생성 파일·거대한 라이브러리 폴더도 제외해 저장소를 깔끔·안전하게 유지합니다.

로컬 저장소 vs 원격 저장소

구분	어디에	동기화
로컬	내 컴퓨터의 .git — 인터넷 없이도 커밋 가능	<code>git push</code> →
원격	GitHub 서버의 Repository	← <code>git pull</code>

주의

두 저장소는 **독립적**입니다. 로컬에서 커밋해도 자동으로 GitHub에 올라가지 않고(반드시 `git push`), GitHub에서 고쳐도 내 컴퓨터에 자동 반영되지 않습니다(반드시 `git pull`). push/pull로 "명시적으로" 맞춥니다.

명령어 git remote add · git push 원격 연결하고 올리기

정의 remote add origin으로 GitHub 주소를 등록(프로젝트당 1회)하고, push로 로컬 커밋을 원격에 올립니다.

TERMINAL

```
git remote add origin https://github.com/사용자이름/저장소이름.git
git remote -v                # 연결 확인
git push -u origin main      # 첫 push (-u는 기본 원격 설정)
git push                     # 이후에는 이것만
```

왜 쓰나 처음 한 번만 -u origin main으로 기본 경로를 설정하면, 이후엔 git push만으로 자동으로 origin main에 올라갑니다.

주의

push 시 인증은 PAT(Personal Access Token)를 사용합니다. 사용자 이름=GitHub 아이디, 비밀번호 자리에 PAT를 붙여넣습니다(진짜 비밀번호 아님).

명령어 git clone · git pull 내려받기 · 최신 반영

정의 clone은 원격 저장소 전체를 내 컴퓨터로 복사(origin 자동 연결), pull은 원격의 최신 변경을 로컬로 가져옵니다.

TERMINAL

```
git clone https://github.com/사용자이름/저장소이름.git
git pull origin main    # 원격 최신 가져오기
git pull                 # 설정 후에는 이것만
```

왜 쓰나 새 컴퓨터에서 프로젝트를 처음 받을 땐 clone, 작업을 시작하기 전엔 pull이 좋은 습관입니다(팀원이 올린 변경을 먼저 받아 충돌을 줄임).

한눈에: 상황별 명령어

이럴 때	이 명령어
내 커밋을 GitHub에 올리고 싶다	git push
GitHub 최신 내용을 내 컴퓨터로 받고 싶다	git pull
GitHub 저장소를 처음 내려받고 싶다	git clone
지금 상태·이력을 확인하고 싶다	git status · git log

개념

GitHub Desktop(선택): 터미널 명령이 부담되면, 클릭으로 Git을 쓰는 무료 앱입니다. 같은 .git을 공유하므로 CLI와 자유롭게 병행할 수 있어요. Changes 체크(=add) → Summary 작성(=commit 메시지) → Commit → Push 버튼. 처음엔 이걸로 감을 잡고, 명령어에 익숙해지면 CLI로 넘어가도 좋습니다.

부록 A

오늘의 명령어·함수 치트시트

파이썬 + Git 한눈에 — 복습·시험용

A-1. 파이썬

분류	이름	형태	한 줄 설명
내장함수	<code>print()</code>	<code>print(a, b)</code>	화면에 출력(심표=공백)
내장함수	<code>str()</code>	<code>str(75)</code>	숫자를 문자로 형변환
내장함수	<code>abs()</code>	<code>abs(-5)</code>	절댓값
자료형	숫자/문자	<code>75</code> vs <code>"75"</code>	따옴표 없으면 숫자
연산자	산술	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>//</code> <code>%</code>	몫// 나머지%
연산자	<code>=</code> vs <code>==</code>	<code>x = 5</code>	저장 vs 비교
문법	f-string	<code>f"{x}"</code>	문장에 값 끼우기
문법	재할당	<code>x = x + 5</code>	오른쪽 먼저 계산 후 저장
문법	다중 할당	<code>x, y = 1, 2</code>	개수 같아야 함
문법	주석	<code># 메모</code>	실행 안 함

A-2. Git 명령어

명령어	무엇	언제
<code>git init</code>	폴더를 저장소로	프로젝트당 1회
<code>git config --global ...</code>	이름·이메일 등록	설치 후 1회
<code>git add .</code>	변경을 Staged로	커밋 전마다
<code>git commit -m "..."</code>	스냅샷 저장	의미 단위마다
<code>git status / log</code>	상태 / 이력 확인	수시로
<code>git remote add origin URL</code>	원격 연결	프로젝트당 1회
<code>git push (-u origin main)</code>	원격에 올리기	커밋 후
<code>git pull</code>	원격 최신 받기	작업 시작 전

명령어	무엇	언제
<code>git clone URL</code>	원격 전체 복사	처음 받을 때

부록 B

오류 사전 & 자주 하는 실수

오늘 코드에서 나오기 쉬운 함정들

B-1. 두 오류만 확실히

오류	무슨 뜻	먼저 확인할 것
<code>NameError</code>	그 이름을 못 찾음(정의 안 됨)	따옴표 누락 · 오타 · 대소문자 · 선언 순서
<code>SyntaxError</code>	문법이 어긋남	따옴표 짝 · 괄호 짝
<code>ZeroDivisionError</code>	0으로 나눔	나누는 값이 0인지

B-2. 오늘 코드의 함정 5가지

X 이렇게 하면	결과	✓ 바로잡기
<code>print(1) vs print("1")</code>	겉보기 같지만 숫자 vs 문자	계산할 값은 따옴표 X
<code>print(0123)</code>	오류(0으로 시작=8진수 취급)	<code>print("0123")</code>
<code>"온도" + 75</code>	문자+숫자 → 오류	<code>"온도" + str(75)</code>
<code>temp vs Temp</code>	서로 다른 변수(대소문자)	이름 철자·대소문자 통일
<code>x, y = 1, 2, 3</code>	개수 안 맞음 → 오류	이름 수 = 값 수

개념

오류가 연쇄로 나도 당황하지 마세요. **한 번에 하나씩만 나타납니다.** 위에서부터 하나 고치고 다시 실행 → 또 나면 또 고치기. 위에서 오류가 나면 그 아래 코드는 실행조차 안 되니, 늘 "맨 위 오류"부터 잡습니다.

부록 C

강의 흐름 연결 & 다음 시간

오늘 배운 것의 지도, 그리고 예고

C-1. 오늘의 세 교안 → 이 책

강의 교안	이 책	핵심
Python 첫 코드와 오류 읽기	STEP 1·3·4	print · 주석 · 오류
변수와 자료형, 연산자와 입력	STEP 2·5	연산자 · 변수(오늘은 여기까지)
Git 기초 + GitHub 연동	STEP 6	버전 관리 전체 흐름

C-2. 다음 시간 예고 (오늘 안 나간 부분)

교안에는 있었지만 오늘 진도로는 안 나간 내용입니다. 미리 이름만 알아두면 다음 시간이 수월합니다.

다음 주제	한 줄 미리보기
자료형과 type()	값의 종류(int·float·str·bool)를 확인하는 함수
비교·논리 연산	== < > 와 and·or — 참/거짓을 다룸
입력과 형변환	input()로 값을 받고 int()·float()로 변환

연결

오늘의 str()에서 "형변환"을 이미 맞췄습니다. 다음 시간의 int()·float()도 같은 원리예요. 숫자↔문자를 오가는 감각을 오늘 코드로 미리 익혀두면 좋습니다.

맺음말 — 오늘의 성취와 다음 한 걸음

오늘 하루로 내 컴퓨터에서 파이썬 파일을 쓰고, 오류를 읽어 스스로 고치고, 변수로 값을 다루고, Git으로 그 기록을 남기는 "개발자의 기본 루프"를 한 바퀴 돌았습니다. 특히 Git/GitHub는 백엔드·임베디드·DX/AX 어느 방향으로 취업하든 실력을 증명하는 포트폴리오가 됩니다. 오늘 만든 파일들을 저장소로 만들어 커밋해두는 것부터가 그 시작이에요.

개념

복습 루틴 추천: ①오늘 만든 `01\_print.py` ~ `04\_variable.py`를 위에서 아래로 다시 실행하며 각 줄이 "코드인지 결과인지, 무슨 분류인지" 말해보기 → ②일부러 오류를 하나 내고 Traceback의 위치·종류·설명 세 가지만으로 고쳐보기 → ③오늘 파일들을 담은 폴더에서 git init → add → commit "study: day2"까지 직접 해보기. 마지막 커밋 한 줄이 오늘의 진짜 마무리입니다.